CLASSIC THREE-TIER AUTO SCALING ARCHITECTURE

ALB + Auto Scaling Group + EC2 Pattern

Complete Setup Guide with Layman Explanations

Production-Grade AWS Deployment Pattern

Interview & System Design Preparation

TABLE OF CONTENTS

1. Overview & Layman Analogy

CLASSIC THREE-TIER AUTO SCALING ARCHITECTURE — The foundational AWS pattern where an ALB distributes traffic to EC2 instances managed by an Auto Scaling Group. Scales up and down automatically based on demand.

Layman Analogy: A Call Center

Think of this setup as a call center. The ALB is the receptionist who routes incoming calls to available agents. The Auto Scaling Group is the HR department that hires new agents when call volume is high and lets some go when it is quiet. The EC2 instances are the agents doing the actual work. The Target Group is the list of agents currently on duty. The Launch Template is the job description that HR uses to hire new agents.

Components at a Glance

Component	What It Is	Analogy
ALB	Application Load Balancer	Receptionist routing calls
ASG	Auto Scaling Group	HR dept (hire/fire agents)
EC2	Virtual servers	Agents doing the work
Target Group	List of healthy instances	List of on-duty agents
Launch Template	Blueprint for instances	Job description for hiring

2. Step 1: Create Launch Template

A Launch Template is the blueprint that tells AWS what kind of EC2 instance to create. It is NOT an instance itself — it is a recipe. The Auto Scaling Group will use this recipe every time it needs to launch a new instance.

Configuration

```
Launch Template:
  Name:            my-app-template
  AMI:             Amazon Linux 2023 (or your custom image)
  Instance Type:   t3.medium
  Key Pair:        my-key
  Security Group:  sg-ec2
                   -> Allow port 8080 from ALB SG ONLY
                   -> Allow SSH (22) from bastion/VPN only
  User Data (startup script):
    #!/bin/bash
    yum install -y java-17
    java -jar /opt/my-app.jar
```

Key Points

- This is a RECIPE, not an instance — no server is created yet
- User Data script runs automatically when each new instance boots
- Security Group should only allow traffic from ALB, not from the internet
- You can version Launch Templates — update without recreating

3. Step 2: Create Target Group (Empty)

IMPORTANT: Create the target group EMPTY. Do NOT add instances manually. The Auto Scaling Group will register instances automatically when they launch.

Configuration

```
Target Group:
  Name:          my-app-tg
  Target Type:   Instance     <-- KEY: not IP (that's for Fargate)
  Protocol:      HTTP
  Port:          8080
  VPC:           my-vpc

Health Check:
  Path:          /actuator/health
  Interval:      30 seconds
  Healthy:       3 consecutive passes
  Unhealthy:     2 consecutive failures

Targets:         EMPTY (do NOT add manually!)
```

Why Instance Type (Not IP)?

With EC2, instances have a fixed identity (Instance ID like i-0abc123). The ASG registers instances by their Instance ID, not by IP address. This is different from Fargate where tasks get dynamic IPs and use the IP target

type.

4. Step 3: Create ALB

The Application Load Balancer sits in public subnets and distributes incoming traffic across healthy instances registered in the target group.

Configuration

```
ALB:
  Name:           my-app-alb
  Scheme:         Internet-facing
  Subnets:        public-subnet-1 (AZ-a)
                  public-subnet-2 (AZ-b)
  Security Group: sg-alb
                  -> Allow port 80 from 0.0.0.0/0
                  -> Allow port 443 from 0.0.0.0/0

Listeners:
  Port 80  -> Redirect to HTTPS (443)
  Port 443 -> Forward to Target Group (my-app-tg)
              SSL Certificate: ACM certificate for myapp.com
```

Security Group Chain

```
sg-alb:   Allow 80, 443 from Internet
sg-ec2:   Allow 8080 from sg-alb ONLY
sg-rds:   Allow 3306 from sg-ec2 ONLY

Internet -> ALB (sg-alb) -> EC2 (sg-ec2) -> RDS (sg-rds)
Each layer only trusts the one before it.
```

5. Step 4: Create Auto Scaling Group

THE ASG TIES EVERYTHING TOGETHER — It uses the Launch Template to create instances and registers them with the Target Group automatically.

Configuration

```
Auto Scaling Group:
  Name:              my-app-asg
  Launch Template:   my-app-template (from Step 1)
  VPC Subnets:       private-subnet-1 (AZ-a)
                     private-subnet-2 (AZ-b)

  Desired Capacity:  3    <-- 'I want 3 instances RIGHT NOW'
  Minimum:           2    <-- 'Never go below 2'
  Maximum:           6    <-- 'Never go above 6'

  Target Group:      my-app-tg  <-- CONNECT HERE!
  Health Check Type: ELB        <-- Use ALB health check
  Health Check Grace: 300 sec   <-- Wait 5 min before checking
```

What Happens When ASG Starts

```
1. ASG reads Launch Template
   'OK, I need t3.medium with Amazon Linux'

2. ASG launches 3 EC2 instances (desired = 3)
```

```
   Instance-1 in private-subnet-1 (AZ-a)
   Instance-2 in private-subnet-2 (AZ-b)
   Instance-3 in private-subnet-1 (AZ-a)

3. ASG AUTOMATICALLY registers them with Target Group
   Target Group: [ Instance-1, Instance-2, Instance-3 ]

4. ALB health checks each instance
   Instance-1: /actuator/health -> 200 OK
   Instance-2: /actuator/health -> 200 OK
   Instance-3: /actuator/health -> 200 OK

5. ALB starts distributing traffic to all 3
```

6. Step 5: Auto Scaling Policies

Option A: Simple Scaling (Manual Rules)

```
Policy 1 - Scale OUT (add instances):
  When: Average CPU > 70% for 5 minutes
  Action: Add 1 instance
  Cooldown: 300 seconds

Policy 2 - Scale IN (remove instances):
  When: Average CPU < 30% for 10 minutes
  Action: Remove 1 instance
  Cooldown: 300 seconds
```

Option B: Target Tracking (Recommended)

```
Target Tracking Policy:
  Metric: Average CPU Utilization
  Target: 50%

  ASG automatically figures out:
    CPU at 80%? -> Add instances to bring it down
    CPU at 20%? -> Remove instances to bring it up
    CPU at 50%? -> Perfect, do nothing

  Much simpler than manual rules!
```

Option C: Scheduled Scaling

```
For PREDICTABLE traffic patterns:

  Every weekday 9 AM:  Set desired = 5 (office hours)
  Every weekday 6 PM:  Set desired = 2 (after hours)
  Every Black Friday:  Set desired = 10 (peak)
```

Scaling Comparison

Type	How It Works	Best For
Simple	You define rules manually	Specific thresholds
Target Tracking	AWS auto-adjusts to target	Most apps (recommended)
Step	Different actions at levels	Graduated response
Scheduled	Time-based scaling	Predictable patterns

7. What Happens After Setup

Complete Request Flow

```
1. User types: https://myapp.com/api/employees

2. Route 53 resolves myapp.com -> ALB IP

3. ALB receives request on port 443
   Checks listener rules -> forward to my-app-tg

4. ALB picks a HEALTHY instance from Target Group
   Using round-robin or least-connections

5. Request forwarded to Instance-2:8080

6. Instance-2 processes, queries RDS, returns response

7. Response: Instance-2 -> ALB -> User
```

Target Group Health Check Flow

```
Every 30 seconds, ALB checks each instance:

  Instance-1: GET /actuator/health -> 200 OK     HEALTHY
  Instance-2: GET /actuator/health -> 200 OK     HEALTHY
  Instance-3: GET /actuator/health -> 500 Error  UNHEALTHY (1/2)

  ... 30 seconds later ...

  Instance-3: GET /actuator/health -> 500 Error  UNHEALTHY (2/2)
  ALB: 'Instance-3 failed 2 checks, removing from rotation'
  Traffic now goes only to Instance-1 and Instance-2

  ASG: 'Instance-3 is unhealthy, terminating...'
  ASG: 'Launching replacement Instance-4...'
  ASG: 'Registering Instance-4 with target group'
```

8. Self-Healing Scenarios

Scenario 1: Instance Crashes

```
ASG: 'Desired = 3, but only 2 running!'
ASG: 'Launching new instance from template...'
ASG: 'Registering with target group...'
ALB: 'New instance healthy, sending traffic'
```

Scenario 2: Instance Fails Health Check

```
ALB: 'Instance-2 failed health check 2 times'
ALB: 'Removing Instance-2 from rotation (no traffic)'
ASG: 'Instance-2 is unhealthy, terminating...'
ASG: 'Launching replacement from template...'
ASG: 'Registering replacement with target group'
```

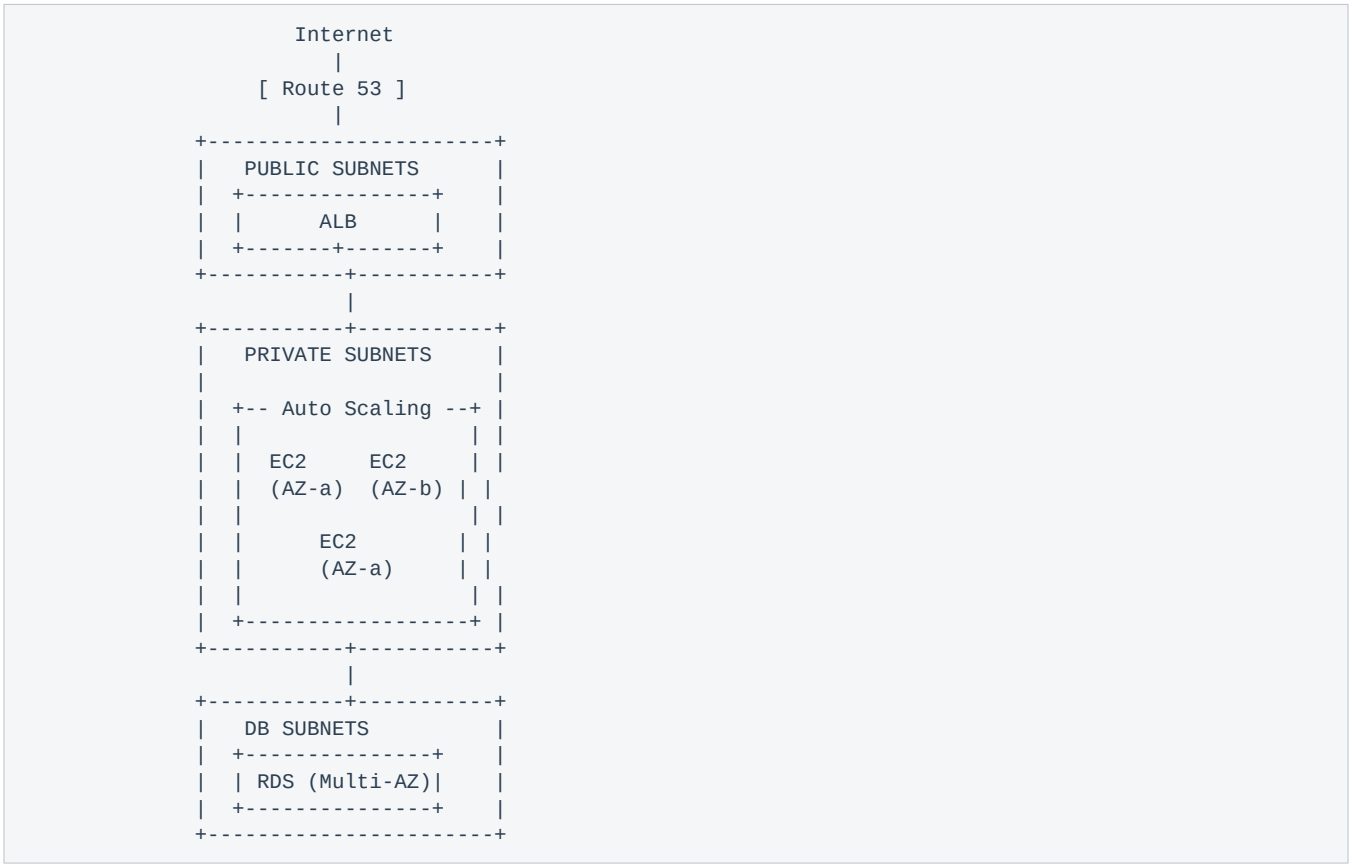
Scenario 3: Traffic Spike

```
CloudWatch: 'CPU > 70% for 5 minutes!'
ASG: 'Scaling out! Launching Instance-4...'
ASG: 'Registering Instance-4 with target group'
ALB: '4 targets now, distributing traffic'
CloudWatch: 'CPU back to 45%. Stable.'
```

Scenario 4: Traffic Drops

```
CloudWatch: 'CPU < 30% for 10 minutes'
ASG: 'Scaling in! But minimum = 2'
ASG: 'Current = 4, can remove instances'
ASG: 'Terminating Instance-4 (newest first)'
ASG: 'Deregistering from target group'
ALB: 'Draining connections, then removing'
ASG: 'Back to 3 instances'
```

9. Complete Architecture Diagram



Subnet Layout Explained

Subnet Type	What Lives Here	Internet Access?
Public Subnets	ALB, NAT Gateway	YES (direct)
Private Subnets	EC2 instances (ASG)	Outbound only (via NAT)
Database Subnets	RDS (Multi-AZ)	NO internet access

10. Target Group: Instance vs IP Type

KEY DIFFERENCE: EC2 + ASG uses **Instance** target type. ECS Fargate uses **IP** target type. This is a common interview question.

Feature	Instance Type (EC2)	IP Type (Fargate)
Registered by	Auto Scaling Group	ECS Service
Identified by	Instance ID (i-0abc...)	IP address (10.0.1.45)
Who manages?	ASG	ECS
Server exists?	YES (can SSH)	NO (serverless)
Scaling speed	Minutes (boot EC2)	Seconds (start container)
Target example	i-0abc123:8080	10.0.1.45:8080

How Registration Works

```
EC2 + ASG (Instance type):
  ASG launches Instance i-0abc123
  ASG calls: register_targets(i-0abc123, port=8080)
  Target Group: [ i-0abc123:8080 ]

ECS Fargate (IP type):
  ECS launches Task with ENI IP 10.0.1.45
  ECS calls: register_targets(10.0.1.45, port=8080)
  Target Group: [ 10.0.1.45:8080 ]

In BOTH cases, registration is AUTOMATIC.
You never add targets manually.
```

11. EC2 + ASG vs ECS Fargate

Feature	EC2 + ASG	ECS Fargate
Target Type	Instance	IP
Who registers?	ASG	ECS Service
Server access?	YES (SSH)	NO (serverless)
Scaling speed	Minutes (boot OS)	Seconds (start container)
Cost model	Pay for full EC2 24/7	Pay per task (vCPU+mem)
OS management	You patch + update	Nothing to manage
Blueprint	Launch Template	Task Definition
Customization	Full OS control	Container only
Best for	Legacy apps, custom OS	Modern containerized apps

When to Use Which

- **Use EC2 + ASG when:** You need full OS access, run legacy apps that cannot be containerized, need GPU instances, or require specific kernel configurations
- **Use ECS Fargate when:** Your app is containerized, you want zero server management, need fast scaling, and prefer a pay-per-use model

12. Interview-Ready Answers

Q: Explain the ALB + ASG + EC2 setup.

"I create a Launch Template as the instance blueprint with AMI, instance type, and startup script. Then I create an empty Target Group with Instance type and health checks. I set up an ALB with listeners that forward to the target group. Finally, I create an Auto Scaling Group that ties everything together — it uses the template to launch instances and automatically registers them with the target group."

Q: How does the target group get populated?

"The target group starts empty. When the Auto Scaling Group launches instances, it automatically registers them with the target group using their Instance IDs. When instances are terminated, ASG deregisters them. I never add or remove targets manually."

Q: What happens when an instance becomes unhealthy?

"The ALB detects the failure through health checks and removes the instance from rotation so no traffic is sent to it. The ASG then terminates the unhealthy instance and launches a replacement from the Launch Template. The new instance is automatically registered with the target group. This self-healing happens without any manual intervention."

Q: Instance vs IP target type?

"EC2 with ASG uses Instance target type because instances have a fixed identity (Instance ID). ECS Fargate uses IP target type because tasks get dynamic IPs through ENIs. In both cases, registration with the target group is automatic — ASG handles it for EC2, and ECS Service handles it for Fargate."

Q: How does scaling work?

"I use Target Tracking auto-scaling policy with a target CPU of 50%. ASG automatically adds instances when CPU exceeds the target and removes them when CPU drops below. I set minimum capacity to 2 for high availability and maximum to 6 as a cost safety net. For predictable patterns, I add scheduled scaling — like scaling up before business hours."